Accelerating Spark SQL with Apache Comet

Native Execution of Analytical Queries in Spark

EC528 Cloud Computing Milestone 2

Members: Pranav Chintareddy, Selena Liu, Adam Yanai, Yongyi Xie, Joseph Attie

Milestone 1 Recap: Problem & System

Motivation

- Big data queries are expensive and complex
- Spark executes operators in JVM
- Need faster execution with better efficiency

System Overview

- SQL ⟶ Spark ⟶ Comet (native execution)
- Data stored in **Parquet (columnar format)**
- Vectorized processing via **Apache Arrow**

Key Idea

- Use **DataFusion Comet** to enable **native execution (Rust)**
- Reduce:
  - JVM overhead
  - Garbage collection cost
  - CPU inefficiency

Milestone 1 Recap: Findings & Challenges

Key Findings

- Comet improves performance significantly
- Up to **~70%+ operators executed natively** (TPC-H)
- Native execution reduces latency

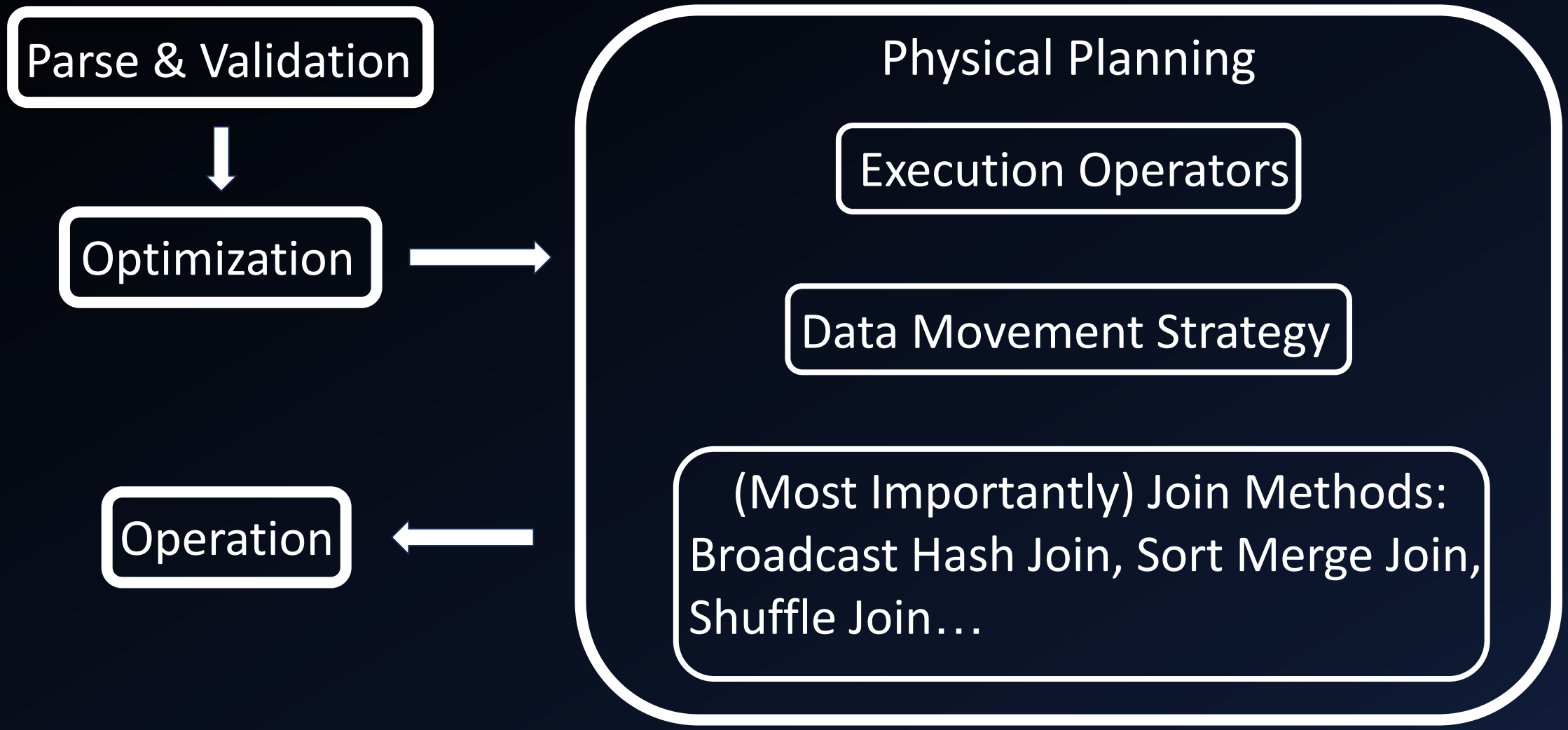
Core Problem: Fallback

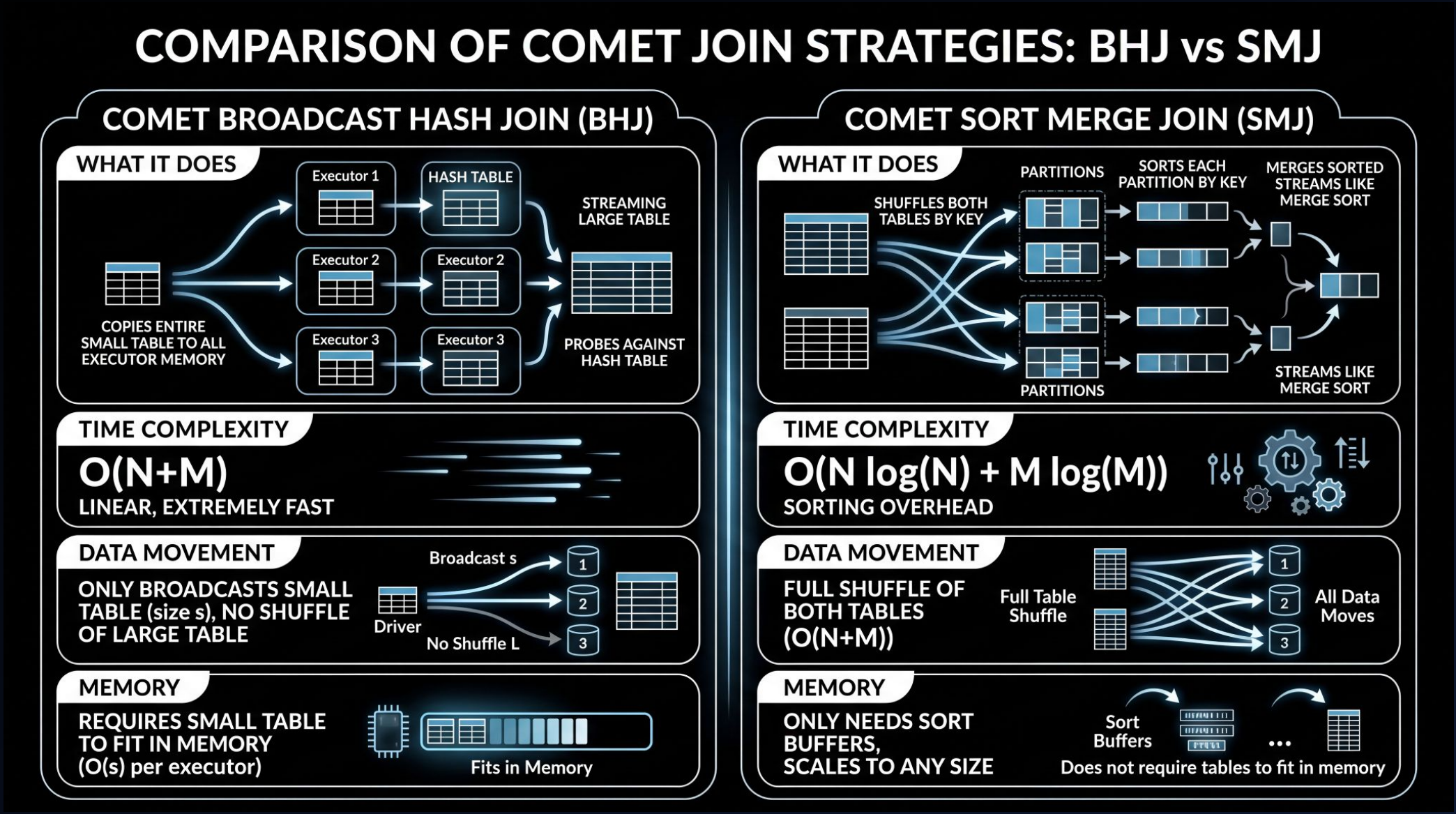
- Not all operators supported in Comet
- Queries may **fallback to Spark (JVM)**

Highlighted Issue:

- **Q21 high latency** (10x slower than Spark execution)
  - Caused by: **Left Semi Join with inequality** (very unoptimized)

Query Execution Pipeline





BHJ is O(n) with minimal data movement but memory-bound.
SMJ is O(n log n) with full shuffle but no memory limit.

Key Problem: Comet Fallback to Spark

- Comet supports limited operators
- Complex SQL → Spark fallback
- Data movement overhead

Fallback breaks the native execution pipeline.

Key Problem: Comet Fallback to Spark

- Comprehensive Tests-Window Based Operations & Joins
- Majority falling back to Spark JVM due to the **ORDER BY** Clause
- Ranking functions(ROW_NUMBER(), RANK() etc) & Value Functions(LEAD(), LAG() etc) completely dependant on the ORDER BY Clause
- Common warning: “COMET: Partitioning and sorting specifications must be the same.”
- ANTI-LEFT JOIN & CROSS JOIN not supported

Comet Warning: BuildRight with LeftAnti is not supported

Issue: Some queries (TPC-H Q16, Q22) used to fall back to Spark for ‘NOT IN’ operator

Root Cause:

- ‘NOT IN’ uses a null-aware anti join — different from regular anti join
- Spark signals this while Comet's Scala layer blocks all BuildRight + LeftAnti joins
- Native DataFusion only implements regular anti join and lacks null-aware semantics

Our Solution:

- Track NULLs on build side – Detect during hash table construction whether any join key contains NULL
- Carry the flag – Pass null-aware distinction from Spark into native execution
- Apply special rules – NULLs on build side suppress output, NULL probe keys are excluded

Result: Native Comet support for both `NOT IN` and `NOT EXISTS` anti joins, Q16 & Q22 have 100% native comet support now

Native Rust support for new Non-benchmark based Operators

- Legacy benchmarks like TPC-H/DS are 15+ years old & don't utilize several modern SQL operations
- Today’s workloads increasingly rely on new cost-efficient aggregate functions like MAX_BY, MIN_BY that currently fall back to JVM
- We implemented:
  - Native support for aggregate functions 'max_by','min_by', w/ full coverage for both HashAggregate and SortAggregate execution paths
  - Native support for 6 mathematical scalar operations: Bitwise Unsigned-Right shift, Factorial, Secant, Cosecant, Cube Root, Hypotenuse

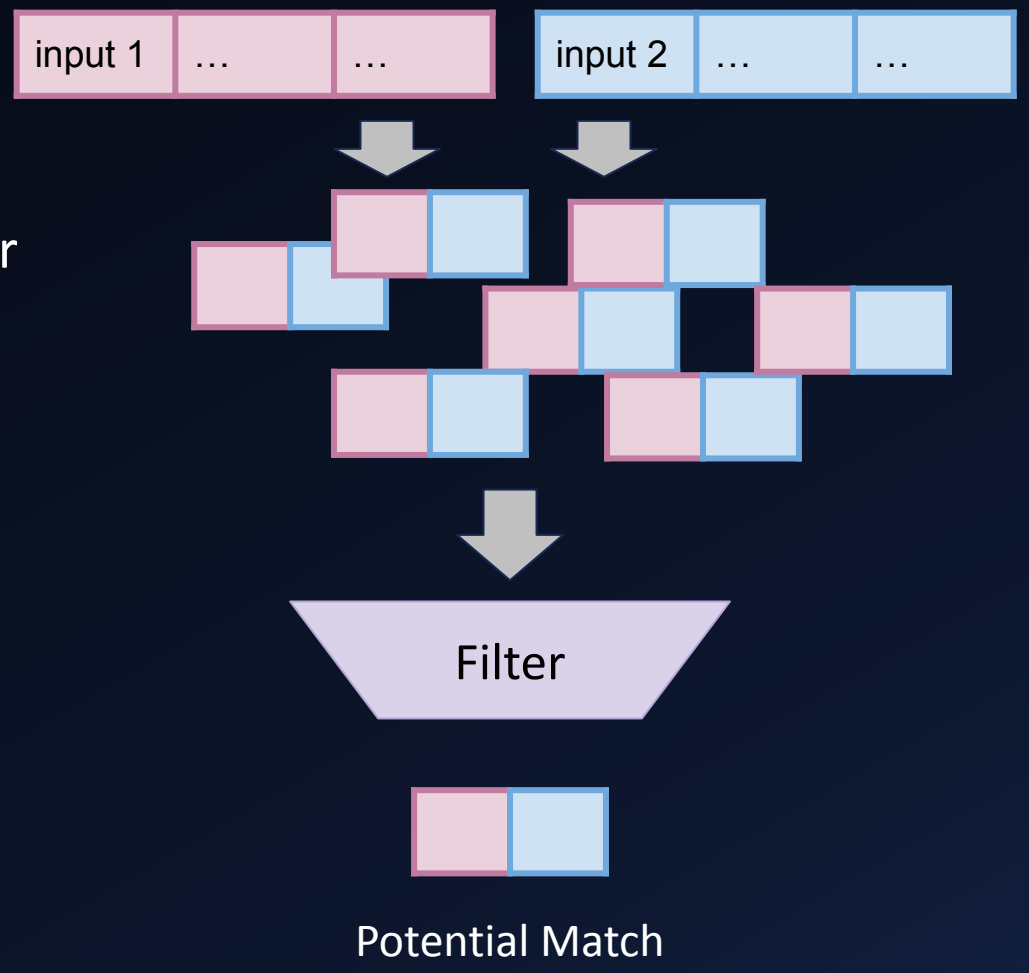
Q21: Where the Current Bottleneck Comes From

- In filtered semi-joins and anti-joins, Comet's SortMergeJoin may
  - Generate many candidate row pairs
  - Then store those pairs + evaluate the filter later

⟶ Unnecessary repeated comparison cost and memory work - **O(N x M)**

- Left Semi Join = “keep row if a match exists”
- Left Anti Join = “keep row if NO match exists”

⟶ Both only need to know if one match exists



A Minimal Example: Equality vs. Inequality

- If the join condition is equality, execution stays efficient
- But changing one = to <> can explode the # of candidate pairs

```
scala> val q_eq = """
     | select l1.l_orderkey, l1.l_suppkey
     | from lineitem l1
     | where exists (
     |   select 1 from lineitem l2
     |   where l2.l_orderkey = l1.l_orderkey
     |     and l2.l_suppkey = l1.l_suppkey
     | )
     | """
q_eq: String =
"
select l1.l_orderkey, l1.l_suppkey
from lineitem l1
where exists (
  select 1 from lineitem l2
  where l2.l_orderkey = l1.l_orderkey
    and l2.l_suppkey = l1.l_suppkey
)
"

scala> spark.sql(q_eq).count   // warm-up
res13: Long = 6001215

scala> val t1 = System.nanoTime
t1: Long = 624352591253816

scala> spark.sql(q_eq).count
res14: Long = 6001215

scala> println("NE secs = " + (System.nanoTime - t1)/1e9)
NE secs = 2.254417736
```

```
scala> val q_ne = """
     | select l1.l_orderkey, l1.l_suppkey
     | from lineitem l1
     | where exists (
     |   select 1 from lineitem l2
     |   where l2.l_orderkey = l1.l_orderkey
     |     and l2.l_suppkey <> l1.l_suppkey
     | )
     | """
q_ne: String =
"
select l1.l_orderkey, l1.l_suppkey
from lineitem l1
where exists (
  select 1 from lineitem l2
  where l2.l_orderkey = l1.l_orderkey
    and l2.l_suppkey <> l1.l_suppkey
)
"

scala> spark.sql(q_ne).count   // warm-up
res23: Long = 5786993

scala> val t1 = System.nanoTime
t1: Long = 624706606372793

scala> spark.sql(q_ne).count
res24: Long = 5786993

scala> println("NE secs = " + (System.nanoTime - t1)/1e9)
NE secs = 29.516249169
```

Optimization Strategy: Filter During the Merge

- Instead of creating + storing all possible row pairs, check the join filter as rows are streamed
- Stop early as soon as the semi / anti-join answer is known
  $\longrightarrow$ Removes unnecessary intermediate pairs while keeping same result

Data Range	Comet enabled w/ Comet SMJ flag	Spark Only (Comet disabled)	**Ours** (Comet enabled + our own SMJ new flag enabled)
1GB	~32 seconds	~3 seconds	~2 seconds
10GB	~2.5 min	~25-28 seconds	~16-18 seconds

Runtime result comparison of Query-21

A 10x speedup!

Key Takeaways

- Physical planning determines performance
- Join strategy is critical
- Avoid fallback to maximize Comet benefits

Thank You!

Quiz Question:
What is the main problem with SortmergeJoinWithFilter that we tried to fix for Q21?

A) The join doesn't use enough threads for parallel processing

B) The join generates and stores all possible row pairs in memory before checking the filter, causing memory churn

C) The join doesn't use indexes correctly

D) The join sorts the data twice